General information

Title : Communication protocol
Eng : Michael
Master description revision : v2 (09/14/2025)

Change sheet

When?	What?
12/02/2025	Created the document and designed a communication protocol.

References

Master description file
https://github.com/DevxMike/kitchen_driver/blob/main/Master_description_file_v2.pdf

General information **1**
Change sheet 1
References 1
Brief **3**
Communication protocol 3
Packet structure 3
Requirements 4
Testing methodology 4

Brief

This document focuses on communication protocol design. Mentioned protocol should be used for exchanging data between various devices that can run C++ (in example: driver and server connected over USB or UART).

Communication protocol

Mentioned protocol should be implemented in a C++ programming language and ensure portability, this implementation should be platform independent. This description will be moved to a separate directory along with implementation.

Packet structure

1) Fields
2) Byte length
3) Valid range
4) Auxiliary range

START	DEV_ID	PACKET TYPE	DATA LEN	DATA	CONTROL SUM	PACKET END
2B	2B	2B	4B	0B - 9999B	4B	2B
0x23; 0x23	0x30 - 0x39	0x30 - 0x39;	0x30 - 0x39;	All ASCII characters from range 0x30 - 0x7E	0x30 - 0x39	0x24; 0x24
		0x41 - 0x5A				

Requirements

REQ_ID	Description
REQ_1	Deserializer should ignore all incoming data if not a start condition (received “##” sequence).
REQ_2	Deserializer should ignore the rest of incoming data if DEV_ID is out of range.
REQ_3	Deserializer should ignore the rest of incoming data if PACKET TYPE is out of range.
REQ_4	Deserializer should ignore the rest of incoming data if DATA LEN is out of range.
REQ_5	Deserializer should invalidate packet if DATA LEN doesn’t match the actual length of received DATA.
REQ_6	Deserializer should invalidate packet if calculated control sum doesn’t match received control sum.
REQ_7	Deserializer should invalidate packet if received packet end is not an END condition (received “$$” sequence).
REQ_8	Serializer and deserializer should use XOR 16bit for data checksum calculation.
REQ_9	Serializer and deserializer should calculate control sum for all fields excluding START and END.
REQ_10	All fields should be represented in ASCII characters.
REQ_11	Deserializer should expose handlers for different types of events (failure and error code, packet received correctly).

Testing methodology

- unit tests - this library should be covered with unit tests (GTEST framework)
- functional tests - this library should be tested against different use scenarios

NOTE: it is assumed, that this protocol is just a baseline for implementation of communication in different systems, integration and qualification tests should be performed agains requirements of a particular system.